

EWA408510 – E-Commerce and Web Application

Final Project Report

Student: Abraham Saryon

Course: EWA408510 – E-Commerce and Web Application

Instructor: Eric Maniraguha

Academic Year: 2025–2026 **Submission Date:**

July, 19, 2026

1. Introduction

AB WebStore is a full-stack e-commerce web application developed as the final project for the course EWA408510 – E-Commerce and Web Application. The platform enables customers in Rwanda and beyond to browse products across multiple categories, manage a shopping cart, place orders, and track their purchases online. The system also includes a fully featured admin dashboard for managing products, orders, customers, reviews, and site settings.

The application was built using modern web technologies including Next.js 14 for the frontend, Node.js with Express.js for the backend API, and MySQL as the relational database. It is fully deployed online, containerized with Docker, and integrated with a CI/CD pipeline using GitHub Actions.

2. Problem Statement

Many local businesses in Rwanda still rely on physical stores and manual processes to sell their products. This limits their reach to local foot traffic and makes it difficult to scale. Customers face challenges such as limited product visibility, no ability to compare prices, and no convenient way to place orders remotely.

There is a clear need for a modern, accessible, and reliable e-commerce platform that allows businesses to list their products online, accept orders digitally, and manage their operations efficiently — while giving customers a seamless and professional shopping experience from any device.

3. Project Objectives

The main objectives of this project are:

1. Design and develop a complete, responsive e-commerce web application
 2. Implement full product management with categories, search, and filtering
 3. Build a functional shopping cart and checkout process with order confirmation
 4. Integrate a secure user authentication system with role-based access control
 5. Develop a comprehensive admin dashboard for store management
 6. Deploy the application online and ensure it remains accessible
 7. Implement a CI/CD pipeline using GitHub Actions for automated builds and testing
 8. Containerize the application using Docker and Docker Compose
 9. Follow best practices in security, code structure, and documentation
-

4. System Features

Customer-Facing Features

- **Homepage** with dynamic hero banner carousel, category navigation, and featured products
- **Product listing** with search by keyword and filter by category, with pagination **Product detail**
- **page** with multiple images, variants (size/color), ratings, and verified reviews
- **Shopping cart** — add, remove, update quantity, auto-calculated totals **Checkout** — shipping
- information form, order summary, payment method selection (Cash on Delivery)
- **Order confirmation page** with order details and status tracking
- **Order history** — customers can view all past orders and their statuses
- **User authentication** — register, login, JWT-based session management **Customer dashboard**
- — profile management, password change, order history, reviews
- **WhatsApp Buy Button** — direct ordering via WhatsApp for selected products
- **Site notice banner** — dismissible announcement bar at the top of every page

Admin Features

- **Dashboard overview** — stats cards (products, orders, revenue, customers), revenue chart (last 30 days), orders by status breakdown
- **Product management** — full CRUD, multi-image upload (drag & drop or URL), product variants, WhatsApp toggle, low stock indicator, search and pagination • **Order management** — view all orders, filter by status, inline status update, perstatus count cards, pagination
- **Customer management** — view profiles with order history, create/edit/delete, suspend/activate accounts, pagination
- **Review moderation** — approve/reject/delete reviews, filter by status, verified purchase badge

- **Site settings** — site name, logo, tagline, hero banners (with reorder), site notice, contact info, social media links
 - **Account settings** — admin profile edit, password change, admin team management
-

5. Technologies Used

Layer	Technology	Purpose
Frontend	Next.js 14	React framework with SSR/SSG support
Styling	Tailwind CSS	Utility-first CSS framework
Backend	Node.js + Express.js	RESTful API server
Database	MySQL 8	Relational database
Authentication	JWT + bcryptjs	Secure token-based auth + password hashing
Image Storage	Cloudinary	Cloud image upload and storage
HTTP Client	Axios	API requests from frontend
Notifications	react-hot-toast	User feedback toasts
Icons	lucide-react	UI icon library
Deployment (Frontend)	Vercel	Serverless Next.js hosting
Deployment (Backend)	Render	Node.js API hosting
Deployment (Database)	Railway	Managed MySQL hosting
Containerization	Docker + Docker Compose	Multi-service containerization
CI/CD	GitHub Actions	Automated build, test, and Docker pipeline
Version Control	Git + GitHub	Source code management

6. System Architecture

The application follows a three-tier architecture:

CLIENT LAYER

Next.js 14 (React) — Vercel

Pages: Home, Products, Cart, Checkout, Orders,
Auth, Dashboard, Admin

HTTPS (Axios)

API LAYER

Node.js + Express.js — Render

Routes: /auth /products /orders /users
/reviews /settings /banners /upload

Middleware: JWT Auth, Admin Guard

mysql2

DATABASE LAYER

MySQL 8 — Railway

Tables: users, categories, products,
orders, order_items, product_images,

product_variants, reviews, banners,
settings

Request Flow:

1. User interacts with the Next.js frontend
2. Frontend sends HTTP requests to the Express API via Axios
3. API validates JWT tokens via middleware
4. Controllers query the MySQL database via mysql2 connection pool
5. Response is returned as JSON and rendered in the UI **Docker Architecture (Local):**

```
docker-compose.yml
db      (mysql:8)      port 3306
backend (node:20-alpine) port 5000
frontend (node:20-alpine) port 3000
```

7. Database Design

The database consists of 10 tables with proper relationships enforced via foreign keys.

Entity Relationship Summary

users orders order_items products

reviews

product_images
product_variants
categories

banners (standalone)
settings (standalone — key/value store)

Table Descriptions

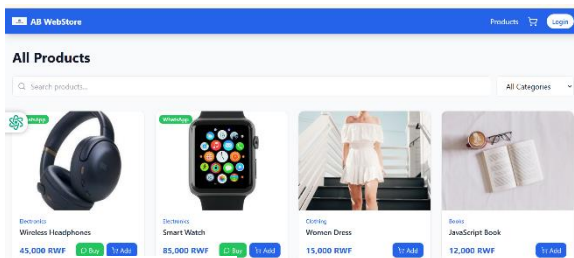
Table	Key Columns	Description
users	id, name, email, password, role, status	Customers and admins
categories	id, name, description	Product categories
products	id, name, price, stock, category_id, whatsapp_enabled	Product catalog
product_images	id, product_id, url, is_primary	Multiple images per product
product_variants	id, product_id, name, value, price_modifier, stock	Size/color variants

orders	id, user_id, total_amount, status, payment_method, shipping_address	Customer orders
order_items	id, order_id, product_id, quantity, price	Line items per order
reviews	id, product_id, user_id, rating, comment, status, verified_purchase	Product reviews
banners	id, title, image_url, active, sort_order	Homepage hero banners
settings	key, value	Site-wide configuration

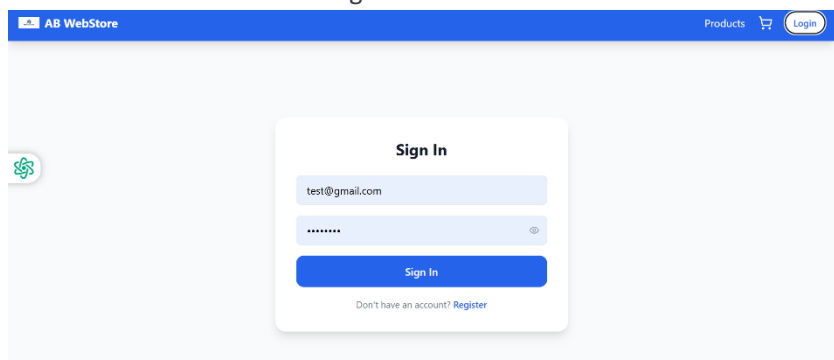
Key Relationships

- products.category_id categories.id (ON DELETE SET NULL) orders.user_id
- users.id (ON DELETE SET NULL) order_items.order_id orders.id (ON DELETE
- CASCADE) order_items.product_id products.id (ON DELETE CASCADE)
- reviews.product_id products.id (ON DELETE CASCADE) reviews.user_id
- users.id (ON DELETE CASCADE) product_images.product_id products.id (ON
- DELETE CASCADE) product_variants.product_id products.id (ON DELETE
- CASCADE)
-

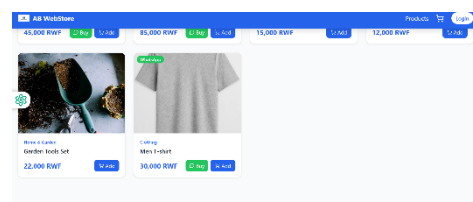
8. Screenshots of the Application



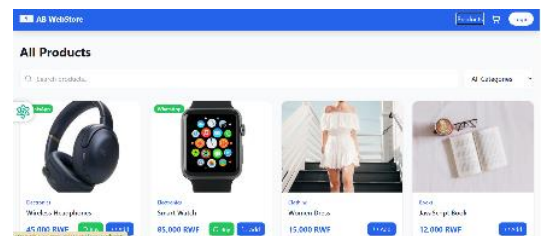
Home Page



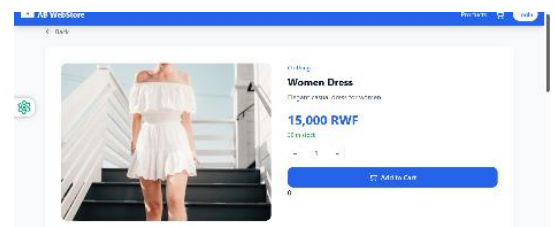
Login



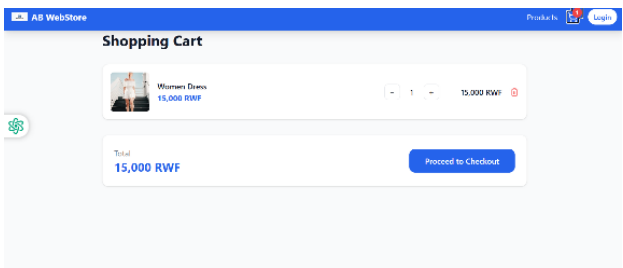
Home Page 2



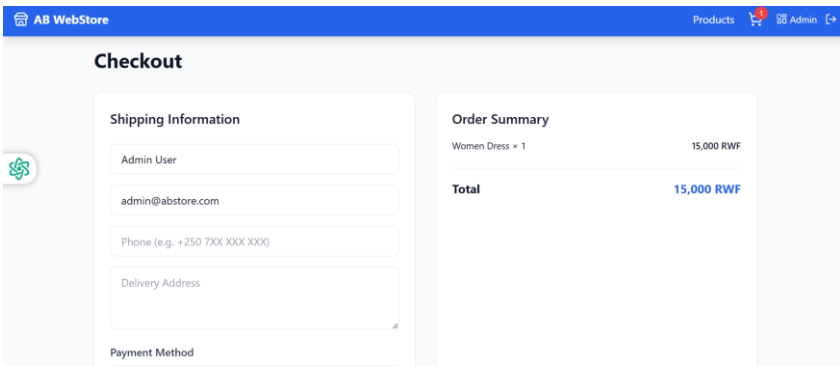
Product-page



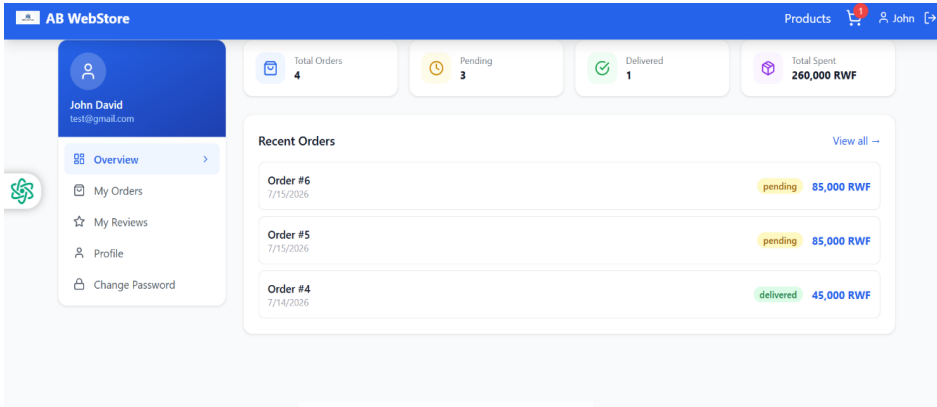
Product-details



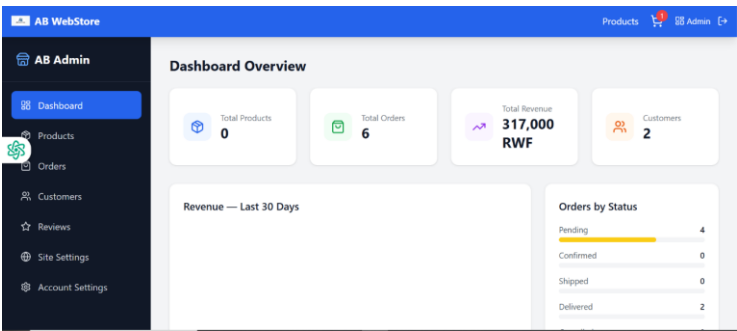
cart



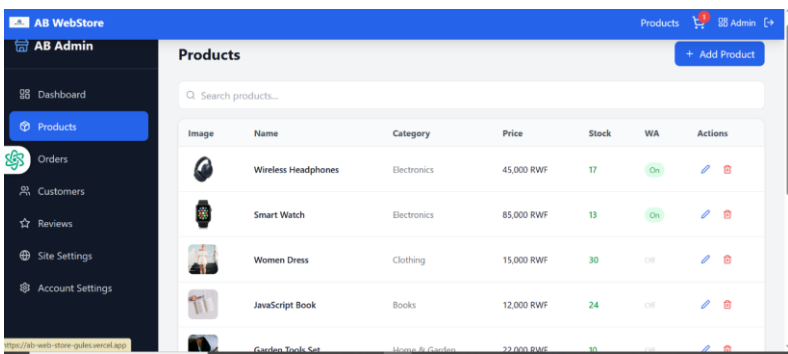
Checkout with order summary



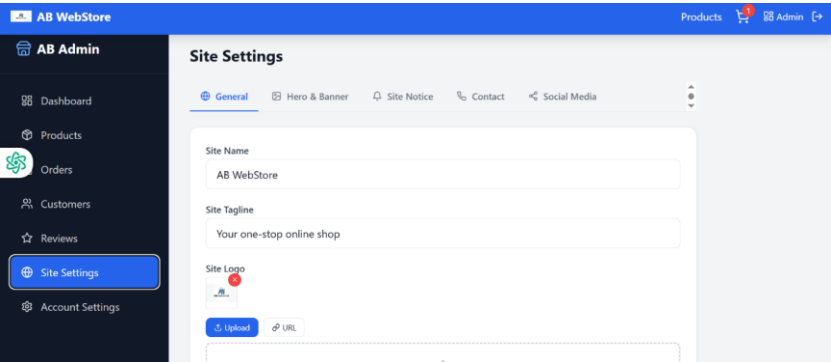
Customer-dashboard



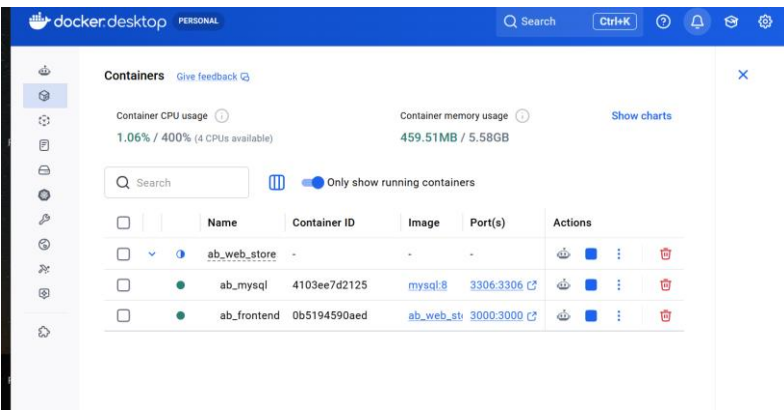
Admin-dashboard



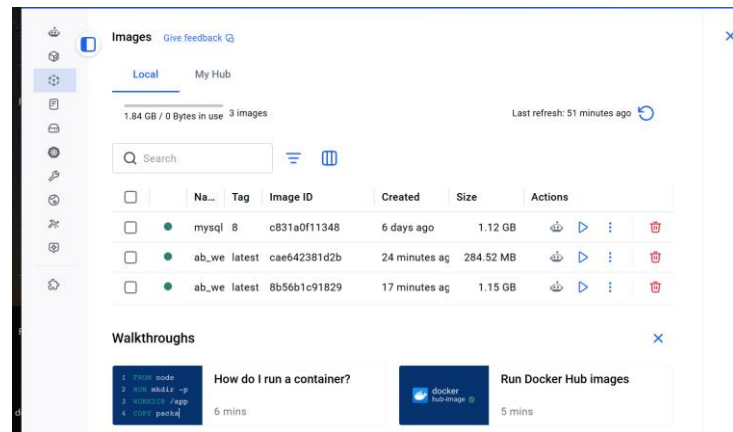
Admin-product



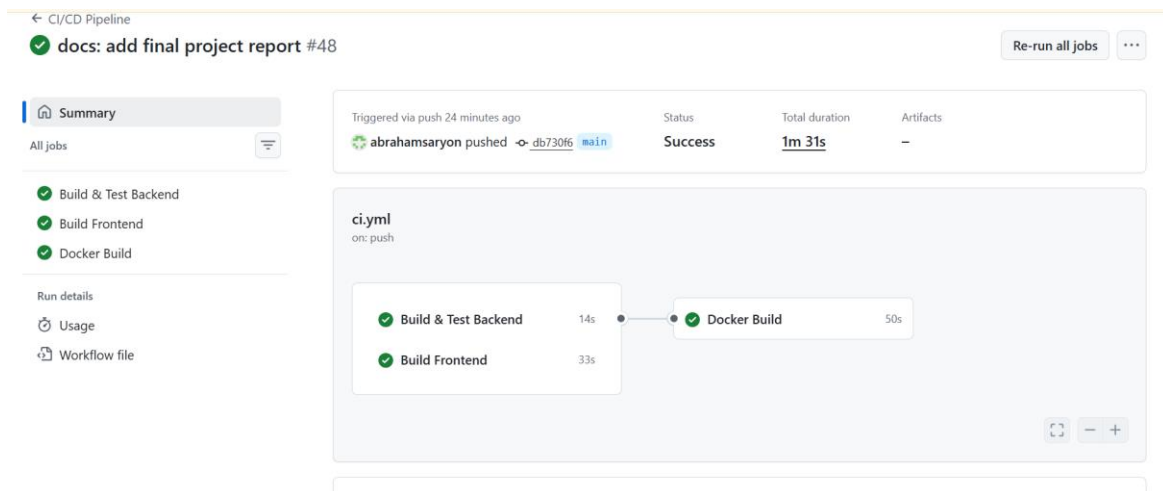
Admin-settings



Docker-container



Docker-image



Github-action

9. GitHub Repository Link

Repository: https://github.com/abrahamsaryon/ab_web_store The

repository contains:

- Full source code for frontend and backend
- README.md with setup instructions and documentation docker-
- compose.yml and individual Dockerfile files
- .github/workflows/ci.yml CI/CD pipeline configuration
- backend/src/config/schema.sql database script .env.example files
- for both frontend and backend
- Meaningful commit history documenting all development stages

10. Deployment Link

Service	URL
Frontend (Live)	https://ab-web-store-gules.vercel.app
Backend API (Live)	https://ab-webstore-backend.onrender.com
Local Frontend	http://localhost:3000 (Docker)
Local Backend API	http://localhost:5000 (Docker)

Deployment Platforms:

- **Vercel** — hosts the Next.js frontend with automatic deployments on every push to main
- **Render** — hosts the Node.js/Express backend API as a web service
- **Railway** — hosts the MySQL database with persistent storage
-

11. CI/CD Implementation

The CI/CD pipeline is implemented using **GitHub Actions** and is defined in `.github/workflows/ci.yml`. It triggers automatically on every push or pull request to the main branch.

Pipeline Stages

Trigger: push / pull_request main

Job 1: Build & Test Backend

Checkout code
Setup Node.js 20
npm install
Start server (smoke test)

Job 2: Build Frontend

Checkout code
Setup Node.js 20
npm install
npm run build (Next.js production build)

Job 3: Docker Build (runs after Jobs 1 & 2)

docker build ./backend
docker build ./frontend

Key Features

- **Parallel execution** — backend and frontend jobs run simultaneously
- **Dependency chain** — Docker build only runs if both build jobs succeed (needs: [test-backend, test-frontend])
- **npm caching** — speeds up subsequent runs using cache: npm
- **Environment variables** — NEXT_PUBLIC_API_URL injected from GitHub Secrets
- **Automatic Vercel deployment** — Vercel auto-deploys on every successful push to main

(Add screenshot of green GitHub Actions workflow run here)

12. Docker Implementation

The application is fully containerized using Docker with a multi-service setup managed by Docker Compose.

Dockerfile — Backend (backend/Dockerfile)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 5000
CMD ["node", "src/index.js"]
```

Dockerfile — Frontend (frontend/Dockerfile)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

Docker Compose (docker-compose.yml)

Three services are defined:

Service	Image	Port	Description
db	mysql:8	3306	MySQL database with schema auto-init
backend	Custom (node:20-alpine)	5000	Express API server
frontend	Custom (node:20-alpine)	3000	Next.js production server

Key features:

- db service mounts schema.sql to /docker-entrypoint-initdb.d/ for automatic database initialization
- backend depends on db , frontend depends on backend (startup order enforced)
- Named volume mysql_data persists database data across container restarts
- All environment variables injected via environment block

Running Locally with Docker

```
git clone https://github.com/abrahamsaryon/ab_web_store.git
cd ab_web_store
docker-compose up --build
```

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:5000>

(Add screenshot of `docker-compose up --build` successful output here)

13. Challenges Encountered

Challenge	Solution
CORS errors between frontend and backend on different domains	Configured Express CORS middleware with <code>FRONTEND_URL</code> environment variable
JWT token expiry causing silent logouts	Added Axios interceptor to catch 401 responses and redirect to login
MySQL connection pooling on Railway with connection limits	Used <code>mysql2</code> connection pool with proper <code>release()</code> in all queries
Next.js hydration errors from server/client mismatch	Added "use client" directives and export <code>const dynamic = "force-dynamic"</code> where needed
Docker virtualization not enabled on development machine	Enabled Hyper-V and WSL2 via PowerShell, updated BIOS settings
SSH push failures to GitHub	Generated new <code>ed25519</code> SSH key, added to GitHub, configured <code>~/.ssh/config</code>
Admin dashboard crash when new stats API not yet deployed	Added graceful fallback with <code>try/catch</code> to derive stats from orders list
Image uploads requiring cloud storage	Integrated Cloudinary for persistent image storage across deployments

14. Future Enhancements

1. **Mobile Money Payment Integration** — MTN Mobile Money and Airtel Money payment gateway for Rwanda
 2. **Real-Time Notifications** — WebSocket-based order status updates pushed to customers
 3. **AI Product Recommendations** — Suggest products based on browsing and purchase history
 4. **Progressive Web App (PWA)** — Offline support and installable app experience
 5. **Multi-Vendor Marketplace** — Allow multiple sellers to list products on the platform
 6. **Advanced Analytics** — Sales trends, top products, customer retention metrics with date range filtering
 7. **Email Notifications** — Automated order confirmation and status update emails
 8. **Inventory Alerts** — Automatic low-stock notifications to admin
 9. **Product Reviews with Images** — Allow customers to upload photos with their reviews
 10. **CSV/PDF Export** — Export orders and customer data for reporting
-

15. Conclusion

AB WebStore successfully demonstrates a complete, production-ready e-commerce web application that meets all the requirements of the EWA408510 final project. The platform covers the full e-commerce lifecycle — from product browsing and cart management to order placement, tracking, and admin management.

The project was built using industry-standard technologies (Next.js, Node.js, MySQL), deployed on professional cloud platforms (Vercel, Render, Railway), containerized with Docker for portability, and integrated with a GitHub Actions CI/CD pipeline for automated builds and deployments.

Beyond the core requirements, the application includes several innovative features such as a WhatsApp ordering system, an analytics dashboard with revenue charts, product variants and multi-image support, admin review moderation with verified purchase enforcement, and dynamic site settings management.

This project has provided valuable hands-on experience in full-stack web development, DevOps practices, database design, cloud deployment, and software engineering principles — skills that are directly applicable to real-world software development careers.
